

Rapport de Sécurisation de Base de Données

Nom : EKPEVI Kossi Augustin

Promotion : B1 Cyber

Année : 2025–2026

Module : BDD112 – Sécurité des bases de données

Projet : PRJ-BDD11

Sommaire

Rapport de Sécurisation de Base de Données	1
Sommaire.....	1
1. Introduction	1
2. Contexte & Objectifs.....	2
Objectifs principaux :	2
3. Présentation du projet existant	2
4. Périmètre fonctionnel minimal	3
5. Contrôle d'accès – RBAC	3
5.1 Rôles applicatifs créés.....	3
5.2 Principes appliqués	3
5.3 Justification (CIA).....	3
6. Row Level Security (RLS)	3
6.1 Exemples de politiques	4
7. Chiffrement	4
7.1 Chiffrement en transit (TLS/SSL)	4
7.2 Chiffrement au champ	4
7.3 Justification.....	4

8. Gestion des secrets & rotation des clés.....	5
9. Audit & journalisation.....	5
9.1 Éléments journalisés	5
9.2 Déclencheurs d’audit	5
9.3 Export & SIEM.....	5
10. Durcissement (Hardening)	5
Mesures appliquées :	6
11. Tests & démonstrations	6
Tests réalisés :.....	6
12. Rapport d’audit & recommandations.....	6
Résultats.....	6
Recommandations	6
Plan d’actions.....	6
13. Conclusion	7
14. Annexes	7
Annexe A – Scripts SQL (RBAC, RLS, Audit).....	7
A.1 Création des rôles RBAC.....	7
A.2 Attribution des privilèges (extraits).....	7
A.3 Activation du Row Level Security	7
A.4 Politiques RLS – Table notes.....	8
A.5 Audit – Table audit_log.....	8
A.6 Trigger d’audit	8
Annexe B – 10 démonstrations de sécurité (pas à pas)	9
Annexe C – Checklist de durcissement (Hardening)	9
Annexe D – Script rotate_keys.sh.....	10
Annexe E – Diagramme RBAC + RLS (description)	10

1. Introduction
2. Contexte & Objectifs
3. Présentation du projet existant
4. Périmètre fonctionnel minimal
5. Contrôle d’accès (RBAC)

6. Row Level Security (RLS)
7. Chiffrement
8. Gestion des secrets & rotation des clés
9. Audit & journalisation
10. Durcissement (Hardening)
11. Tests & démonstrations
12. Rapport d'audit & recommandations
13. Conclusion
14. Annexes

1. Introduction

Ce rapport s'inscrit dans le cadre du module **BDD112 – Sécurité des bases de données**, suivi en première année de Bachelor Cyber. Il a pour objectif de présenter la mise en œuvre complète des mécanismes de sécurité appliqués à une base de données PostgreSQL existante, développée lors du module BDD111.

La sécurité des données étant un enjeu majeur en cybersécurité, ce projet met en pratique des notions fondamentales telles que le contrôle d'accès, le chiffrement, l'audit et le durcissement, conformément aux bonnes pratiques professionnelles.

2. Contexte & Objectifs

Le projet PRJ-BDD11 représente une base de données de gestion scolaire (élèves, parents, enseignants, notes, absences, messagerie). Cette base contient des données sensibles nécessitant un haut niveau de protection.

Objectifs principaux :

- Mettre en place une authentification par rôles applicatifs.
- Implémenter un contrôle d'accès granulaire (RBAC + RLS).
- Sécuriser les données par chiffrement en transit et au repos.
- Assurer la traçabilité des actions via l'audit et la journalisation.
- Appliquer une politique de durcissement (hardening).

3. Présentation du projet existant

La base existante repose sur PostgreSQL et comprend notamment les tables suivantes :

- utilisateur
- etudiant
- parent
- professeur
- note
- absence
- messagerie
- Cantine
- Classe
- Journal (document)
- sanction
- aller (table de jonction)
- responsabiliser (table de jonction)
- emploi_du_temps
- etablissement
- devoir
- être (table de jonction)

Ces tables manipulent des données personnelles et académiques, ce qui justifie la mise en place de mécanismes de sécurité avancés.

4. Périmètre fonctionnel minimal

Le périmètre défini par le cahier des charges impose :

- La création de rôles applicatifs distincts.
- L'activation du Row Level Security sur au moins 3 tables sensibles. (table étudiant ,messagerie,,utilisateur,, parent, personnel ,professeur)
- Le chiffrement des données sensibles. (les noms et prenom , les messages ...)
- La mise en place d'un audit détaillé.
- Une checklist de durcissement complète.

5. Contrôle d'accès – RBAC

5.1 Rôles applicatifs créés

Les rôles suivants ont été définis :

- app_read
- app_write
- app_student
- app_parent
- app_teacher
- app_admin

5.2 Principes appliqués

- Principe du moindre privilège.
- Séparation stricte des droits.
- Interdiction de connexion directe (NOLOGIN) pour les rôles applicatifs.

5.3 Justification (CIA)

- **Confidentialité** : accès limité aux données nécessaires.
- **Intégrité** : seuls les rôles autorisés peuvent modifier les données.
- **Disponibilité** : accès contrôlé mais fonctionnel pour chaque acteur.

6. Row Level Security (RLS)

Le RLS a été activé sur les tables suivantes :

- notes
- messages
- absences

6.1 Exemples de politiques

- Un élève ne peut consulter que ses propres notes.
- Un parent ne voit que les absences de ses enfants.
- Les messages sont visibles uniquement par les participants.

Chaque table possède :

- SELECT POLICY
- INSERT POLICY (si nécessaire)
- UPDATE POLICY
- DELETE POLICY (rare et justifiée)

7. Chiffrement

7.1 Chiffrement en transit (TLS/SSL)

- Activation du SSL côté PostgreSQL.
- Utilisation de certificats auto-signés.
- Stockage sécurisé dans `/docker/ssl/`.

7.2 Chiffrement au champ

L'extension `pgcrypto` est utilisée pour chiffrer :

- Adresses email
- Messages privés
- URI de pièces jointes
- Données sensibles

7.3 Justification

Le chiffrement au champ permet de protéger les données même en cas d'accès non autorisé à la base.

8. Gestion des secrets & rotation des clés

Les clés de chiffrement sont stockées hors du code source.

Un script `rotate_keys.sh` permet :

- La génération de nouvelles clés.
- Le rechiffrement automatique des données.

Cette rotation réduit l'impact d'une compromission éventuelle.

9. Audit & journalisation

9.1 Éléments journalisés

- Connexions et déconnexions.
- Requêtes sensibles (INSERT, UPDATE, DELETE).
- Modifications sur les tables critiques.

9.2 Déclencheurs d'audit

Des triggers AFTER INSERT/UPDATE/DELETE enregistrent les événements dans des tables d'audit.

9.3 Export & SIEM

- Export des logs en JSON/CSV.
- Intégration avec un mini pipeline SIEM (OpenSearch / Wazuh).

10. Durcissement (Hardening)

Mesures appliquées :

- Protection des fichiers .key.
- Restriction des droits sur les schémas.
- Désactivation des fonctions dangereuses.
- Limitation des extensions PostgreSQL.
- Interdiction de login pour les rôles applicatifs.

Une checklist complète a été réalisée et validée.

11. Tests & démonstrations

Tests réalisés :

- 10 démonstrations de sécurité (RBAC, RLS, erreurs attendues).
- 6 tests unitaires SQL simulant plusieurs utilisateurs.
- 4 vues sécurisées :
 - Vue masquée (partial reveal).
 - Vue réservée à l'administrateur.

Les tests ont été effectués via SET ROLE.

12. Rapport d'audit & recommandations

Résultats

- Aucun accès non autorisé détecté.
- Politiques RLS fonctionnelles.

Recommandations

- Automatiser davantage la rotation des clés.
- Ajouter des alertes temps réel côté SIEM.

Plan d'actions

Les actions sont priorisées selon leur criticité (haute, moyenne, faible).

13. Conclusion

Ce projet m'a permis de comprendre concrètement les enjeux de la sécurité des bases de données. En tant qu'étudiant en première année de cybersécurité, j'ai pu appliquer des concepts théoriques à un cas réel, proche des pratiques professionnelles.

Les notions de moindre privilège, de chiffrement et d'audit constituent une base essentielle pour ma future carrière dans la cybersécurité.

14. Annexes

Annexe A – Scripts SQL (RBAC, RLS, Audit)

A.1 Création des rôles RBAC

```
CREATE ROLE app_read NOLOGIN;  
CREATE ROLE app_write NOLOGIN;  
CREATE ROLE app_student NOLOGIN;  
CREATE ROLE app_parent NOLOGIN;  
CREATE ROLE app_teacher NOLOGIN;  
CREATE ROLE app_admin NOLOGIN;
```

A.2 Attribution des privilèges (extraits)

```
GRANT SELECT ON notes, absences TO app_teacher;  
GRANT SELECT ON notes TO app_student;  
GRANT INSERT, UPDATE ON notes TO app_teacher;  
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO app_admin;
```

A.3 Activation du Row Level Security

```
ALTER TABLE notes ENABLE ROW LEVEL SECURITY;  
ALTER TABLE absences ENABLE ROW LEVEL SECURITY;  
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;
```

A.4 Politiques RLS – Table notes

```
CREATE POLICY notes_select_student  
ON notes  
FOR SELECT  
TO app_student  
USING (student_id = current_setting('app.current_student')::int);  
  
CREATE POLICY notes_select_teacher
```

```
ON notes
FOR SELECT
TO app_teacher
USING (true);
```

A.5 Audit – Table audit_log

```
CREATE TABLE audit_log (
    id SERIAL PRIMARY KEY,
    username TEXT,
    action TEXT,
    table_name TEXT,
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

A.6 Trigger d'audit

```
CREATE OR REPLACE FUNCTION audit_trigger()
RETURNS trigger AS $$
BEGIN
    INSERT INTO audit_log(username, action, table_name)
    VALUES (current_user, TG_OP, TG_TABLE_NAME);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER audit_notes
AFTER INSERT OR UPDATE OR DELETE ON notes
FOR EACH ROW EXECUTE FUNCTION audit_trigger();
```

Annexe B – 10 démonstrations de sécurité (pas à pas)

1. Connexion en étudiant → accès autorisé à SES notes

```
SET ROLE app_student;
SELECT * FROM notes; -- filtré par RLS
```

2. Étudiant tente de voir toutes les notes → refus logique RLS
3. Étudiant tente UPDATE → refus

```
UPDATE notes SET grade = 20; -- ERROR
```

4. Professeur consulte toutes les notes → OK
5. Parent tente SELECT global → refus
6. Admin accès total → OK
7. Accès message hors conversation → refus
8. Insertion absence par prof → OK
9. Suppression absence par étudiant → refus
10. Vérification log audit après modification

```
SELECT * FROM audit_log;
```

Annexe C – Checklist de durcissement (Hardening)

- SSL/TLS activé
- Certificats stockés hors code
- Rôles applicatifs NOLOGIN
- Principe du moindre privilège
- RLS activé sur tables sensibles
- Extensions limitées (pgcrypto uniquement)
- Fonctions dangereuses désactivées
- Permissions schémas restreintes
- Logs activés
- Audit via triggers

Annexe D – Script rotate_keys.sh

```
#!/bin/bash
```

```
OLD_KEY=$(cat /secure/old.key)
```

```
NEW_KEY=$(openssl rand -base64 32)
```

```
echo "$NEW_KEY" > /secure/new.key
```

```
psql -U admin -d schooldb <<EOF
UPDATE users
SET email = pgp_sym_encrypt(
    pgp_sym_decrypt(email::bytea, '$OLD_KEY'),
    '$NEW_KEY'
);
EOF

mv /secure/new.key /secure/active.key
```

Annexe E – Diagramme RBAC + RLS (description)

- Utilisateur → rôle applicatif
- Rôle → privilèges SQL
- RLS filtre les lignes selon l'identité

Exemple :

Élève → app_student → SELECT notes → RLS (student_id)

Ce schéma garantit confidentialité et moindre privilège.

- Scripts SQL
- Script rotate_keys.sh
- Captures d'écran
- Checklist de durcissement
- Documentation technique PostgreSQL

Captures d'écran des testes :

Mon MCD :

List of relations			
Schema	Name	Type	Owner
edusys	absence	table	edusys
edusys	aller	table	edusys
edusys	audit_log	table	edusys
edusys	cantine	table	edusys
edusys	classe	table	edusys
edusys	devoir	table	edusys
edusys	emploi_du_temps	table	edusys
edusys	etablissement	table	edusys
edusys	etre	table	edusys
edusys	etudiant	table	edusys
edusys	journal	table	edusys
edusys	messagerie	table	edusys
edusys	note	table	edusys
edusys	parent	table	edusys
edusys	personnel	table	edusys
edusys	professeur	table	edusys
edusys	responsabiliser	table	edusys
edusys	sanction	table	edusys
edusys	utilisateur	table	edusys

La table absence

id_absence	h_debut	h_fin	motif	id_etu	id_prof
1	08:40:00	17:50:00	Rendez-vous médical	99	11
2	12:30:00	17:30:00	Non justifié	69	2
3	10:30:00	09:30:00	Rendez-vous médical	65	13
4	16:50:00	17:20:00	Justifié	43	26
5	16:40:00	09:30:00	Grève transports	58	5
6	09:50:00	15:20:00	Justifié	75	13
7	10:00:00	13:30:00	Rendez-vous médical	81	9
8	12:20:00	10:20:00	Rendez-vous médical	66	15
9	13:20:00	10:40:00	Justifié	88	27
10	13:40:00	15:10:00	Justifié	23	5
11	08:10:00	11:50:00	Grève transports	40	19
12	09:00:00	16:20:00	Non justifié	52	10
13	15:00:00	09:50:00	Non justifié	44	12
14	11:30:00	15:20:00	Rendez-vous médical	38	19
15	16:20:00	09:10:00	Malade	14	11
16	08:30:00	09:40:00	Justifié	100	8
17	10:00:00	10:50:00	Grève transports	84	21
18	09:40:00	12:30:00	Rendez-vous médical	26	6
19	15:50:00	16:30:00	Malade	14	21
20	09:10:00	10:30:00	Non justifié	1	13
21	10:50:00	11:10:00	Rendez-vous médical	38	21
22	12:30:00	09:20:00	Justifié	8	14
23	09:10:00	14:20:00	Malade	72	17
24	12:40:00	14:40:00	Grève transports	22	12
25	15:20:00	12:00:00	Non justifié	49	16
26	15:30:00	13:50:00	Malade	22	12
27	09:20:00	15:10:00	Non justifié	68	21

La table utilisateurs

id_utilisateur	mail
mail_hash	mot_de_passe
1	\xc30d040703022bbb9ef2ef408ec272d24501e74252a826be7b0c1c4c60edf14d57fdf5ad8b90a379a4d62273724de15d9b4a
2	\xc30d040703022008d67766e5d9b265d244015849928550d29ef266faaa9cd47551e4f15c6ce905b6711144bc978115b469ab
3	\xc30d04070302bfb89a784f9c1b29062d245015aa46423940018091118ea8862c75bacc6a628c77e99b9cb66236ee214d53b96
4	\xc30d0407030291af5d116e677c4f7cd243017166adddbb6d720125287700c2204d64835070e93afc1ea631be3098d6808d1e
5	\xc30d04070302c112d78aebad0967d246015915c99f5107d8a6ada38ca9ad5632d03957383d1534752a01c27ec543c6300b
6	\xc30d04070302df8bba8d9f778a861d2490189588809e3d166b18bc68f5806f1d75a35d4e2ebb4d13935111b3cfff3ede586
7	\xc30d0407030282883622cce9809a76d24e01c9582bdcd008bb49fa116619e493b764d78d14a84b539268a5ad38ebfe40ce94
8	\xc30d04070302aa9d815224fd40d76fd24e01bd53e49da012121b9a9902936961fefbd85ccedd7f60a4b0319120c1651d20a4c4

La table etablisement

id_eta	nom	adresse	mail	mail_hash
1	Lombard S.A.S.	8, rue Élodie Hubert		\xc30d04070302395ea0eee6f2d06770d24401fe9b318f1431918bbf152e6c8715e414d6a1ca103f316ce572d46d62953b83963b26e19344ebb6fc3b2c17a7f07ef34e8ad3faf5aaf0d3fa576695389687821cb4ccce4b0f853162c898c79d127474353d498656da7574f3c82e8c22e7eb57ca1097d5
		13377 MaréchalVill		
2	Girard	105, boulevard Agathe Cohen		\xc30d040703022ff016b0a200fdc72d24b01b6ce1d474bb42f3b25939f11eb067c6a47b040904046897e027c8c322ec4559173b918dd6244554b038e1c14af8e268b45b02c198f557cb6d1bb337b84143df58dd3e7a8bf0f3f56956f8a141c80b6a805080fb78895e094ee9cfc3e4b32bf9896952e9c3d5d49db7cd5
		46501 Saint MargotVill		
3	Briand S.A.S.	78, boulevard Joseph Delahaye		\xc30d04070302036f0b19ed71a83172d24a01631a613c5e5768d7dfd691b35f8565370300a07c444f452774633f455184e60259347f481490db474b09ae620b872b91794c951e866124da831a585ebf578e0295c7ff26b2ad14b2c08b7ade494f14076771181959076ad10bf62577c30bf90daf24b2ef58a4d43e3
		97279 Chevalier		
4	Petit	boulevard Bernard Marion		\xc30d0407030247acd715aeeac48d61d2450181e51e753b83eb0921e597c4d0090e9cc2cf73029a1a38b882508935e7a73aa409f399c6a458b112cc0a9ef28a99ab7334a168e75b32ccd32ed863d1ecd539a021095f54a8edb6b37cd9650c254dc5815aa452127039f9abbdda25ad6a77645e185a5050
		43699 Benardnec		

Table cantine

id_cantine	prix	nb_repas	id_etu
1	\$4.07	2	81
2	\$5.05	18	77
3	\$5.92	20	13
4	\$7.41	22	75
5	\$7.01	18	43
6	\$4.08	24	83
7	\$3.91	16	99
8	\$5.76	4	53
9	\$7.25	30	15
10	\$6.75	10	87
11	\$6.53	12	5
12	\$7.21	11	59
13	\$4.57	18	45
14	\$6.26	9	59
15	\$4.87	8	78
16	\$5.51	5	10
17	\$4.45	29	80
18	\$6.65	24	25
19	\$5.98	27	53
20	\$7.84	21	80
21	\$4.10	11	77
22	\$4.28	15	58
23	\$5.54	16	97
24	\$7.58	16	72
25	\$6.65	24	19
26	\$6.88	17	14
27	\$5.43	30	91

id_classe	nom_classe
1	6ème A
2	6ème B
3	5ème A
4	5ème B
5	4ème A
6	4ème B
7	3ème A
8	3ème B
9	2nde A
10	2nde B
11	Terminale S
12	Terminale ES
13	1ère S
14	1ère ES
15	1ère L

1. Vérifier que les noms sont chiffrés (données binaires)

:

Commande : `docker exec -it edusys-db psql -U edusys -d edusys -c "SELECT id_etu, nom, prenom FROM edusys.etudiant LIMIT 5;"`

	id_etu	nom	prenom
1	\xc30d04070302af6da17fcf78870d7ed235017ff66e7b8a553d91cce6f33abd7ca2fcd871791441ba42cb0b8f6cdff3946a1121f36e1964a3582deb01b26ddcbe39c1baef38e2		
2	\xc30d04070302a600317d50558e5867d237013f5d429c02018d93468681b8081b9fbf13e6c0b06ac640beed154636cb46916f3e68b734fd11dde9b6910fd6e4857aa2d271a811cbbc		
3	\xc30d0407030280fb2302c16d5c8a74d237014fd15c3614bf33b43a9b9425ca7b0f6bda5129788f2e92039537b97d02669ebc262e4bee20285621e6a94584d16ded6736571d8504ca		
4	\xc30d04070302e23f1a7dd0d580816ad23a01f25aa30e792c00499a7ddea679ad47b91eba0d6e13517b91466431379d90a55abf612be81c5d01f52296e615c02f09d4168d27c1471b4da9		
5	\xc30d04070302a30b7b2a0e7fe88770d23701ec9378de66b198ac96647c73b10596e765f4792f3d9cc552f62032f0b978d62251de8929641aa3876abf9007369e61d552258b88268f		

2. Vérifier le déchiffrement via la vue sécurisée

Commande : `docker exec -it edusys-db psql -U edusys -d edusys -c "SELECT * FROM edusys.vue_etudiant_masque LIMIT 5;"`

nom	prenom_masque	id_classe
Roux	Z***	14
Dupont	J***	15
Letellier	A***	11
Buisson	V***	4
Joseph	T***	2
(5 rows)		

3. Lancer le rapport de sécurité complet (Scripts de tests)

Commande : `Get-Content tests/demo_security.sql | docker exec -i edusys-db psql -U edusys -d edusys`

TEST 1: RBAC - Simulation connexion Student 1

```
--- TEST 1: RBAC - Simulation connexion Student 1 ---
SET
SET
  current_user | current_setting
-----+-----
  app_student | 1
(1 row)
```

TEST 2: RLS - Student 1 consulte son profil

```
--- TEST 2: RLS - Student 1 consulte son profil ---
id_etu |                                nom                                |
-----+-----
      1 | \xc30d04070302af6da17fcf78870d7ed235017ff66e7b8a553d91cce6f33abd7ca2fcd871791441ba42cb0b8f6cdf3946a1121f36e1964a3582deb01b26ddcbe39c1baef38e2 | \xc30d04070302e7ce98653a79558366d2390110ef849ab1176613b3c992954e77be83e46c5c91af4b6ac1e807ecb0cfde7be0bad6eda6a35823b772801c278bfef08c747de33177ecdd5a
(1 row)
```

TEST 3: RLS - Student 1 essaie de voir tous les ??tudiants (Count doit ??tre 1)

```
--- TEST 3: RLS - Student 1 essaie de voir tous les ??tudiants (Count doit ??tre 1) ---
count
-----
      1
(1 row)
```

TEST 4: RLS - Student 1 consulte ses notes

```
--- TEST 4: RLS - Student 1 consulte ses notes ---
```

id_note	matiere	matiere_name
50	0.09	EPS
137	0.95	Histoire-Géographie
253	0.20	Français
276	0.76	Mathématiques
499	0.80	Sciences
629	0.66	Arts
646	0.85	Sciences
648	0.69	Français
653	0.22	Français
698	0.36	Physique-Chimie
909	0.14	Histoire-Géographie
1018	0.22	Informatique
1094	0.66	Anglais
1366	0.35	Physique-Chimie
1385	0.71	Informatique
1489	0.64	Sciences
1493	0.15	Anglais
1530	0.75	Histoire-Géographie
1594	0.05	Informatique
1604	0.73	Histoire-Géographie
2000	0.00	Anglais
2001	0.50	Test Audit
2002	0.50	Test Audit
2030	0.76	SVT
2241	0.45	Physique-Chimie
2282	0.88	Physique-Chimie
2353	0.24	Histoire-Géographie
2379	0.30	Histoire-Géographie
2384	0.95	Sciences
2385	0.49	Français
2489	0.30	Français
2545	0.88	Arts
2613	0.16	Histoire-Géographie
2810	0.70	Physique-Chimie
2847	0.37	Informatique
3039	0.64	EPS

TEST 5: RBAC - Student essaie de modifier une note (Doit échouer)

```
--- TEST 5: RBAC - Student essaie de modifier une note (Doit échouer) ---
DO
```

TEST 6: Simulation connexion Parent 1

```

--- TEST 6: Simulation connexion Parent 1 ---
NOTICE: Succ??s : Modification interdite pour Student (Erreur attendue)
RESET
SET
SET

```

Parent consulte les notes de son enfant :

id_note	matiere	matiere_name
26	0.46	Informatique
50	0.09	EPS
55	0.25	Français
69	0.12	Physique-Chimie
70	0.21	Histoire-Géographie
137	0.95	Histoire-Géographie
215	0.91	Histoire-Géographie
217	0.55	Histoire-Géographie
253	0.20	Français
270	0.56	Physique-Chimie
276	0.76	Mathématiques
287	0.33	Informatique
337	0.67	SVT
359	0.19	Physique-Chimie
398	0.63	Sciences
405	0.55	Physique-Chimie
412	0.31	Anglais
452	0.36	Mathématiques
462	0.62	Anglais
472	0.05	Arts
474	0.91	Français
499	0.80	Sciences
502	0.66	EPS
523	0.27	Arts
566	0.44	Mathématiques
629	0.66	Arts
646	0.85	Sciences
648	0.69	Français
653	0.22	Français
659	0.60	Français

TEST 7: Encryption - Lecture email

```

-----+-----
235 | \xc30d0407030298a5af28d5643e486ad243012378169265ca25a62d6faa31098160c8cf08df8e49ad23ca7205388aa7d1700b0543378aa196edd49818eed32bb17e3789a7f15
f2bd6e6061515720c73988cd85f6
(1 row)

```

TEST 8: Audit - Prof modifie une note

```

--- TEST 8: Audit - Prof modifie une note ---
RESET
SET
SET
INSERT 0 1

```

TEST 9: Vérification Audit Log (Admin)

```

--- TEST 9: Vérification Audit Log (Admin) ---
RESET
SET

```

table_name	operation	user_name	changed_fields
note	INSERT	edusys	(null)

(1 row)

TEST 10: Vue Masquée (Partial Reveal)

```

--- TEST 10: Vue Masquée (Partial Reveal) ---
SET

```

nom	prenom_masque	id_classe
Roux	Z***	14
Dupont	J***	15
Letellier	A***	11
Buisson	V***	4
Joseph	T***	2

(5 rows)

4. Vérifier les logs d'Audit

Vérifiez que les actions (comme les insertions du seeder) ont bien été enregistrées.

Commande : `docker exec -it edusys-db psql -U edusys -d edusys -c "SELECT table_name, operation, timestamp FROM edusys.audit_log LIMIT 10;"`

table_name	operation	timestamp
messagerie	INSERT	2026-01-05 12:17:40.86457
messagerie	INSERT	2026-01-05 12:17:40.871011
messagerie	INSERT	2026-01-05 12:17:40.875617
messagerie	INSERT	2026-01-05 12:17:40.879219
messagerie	INSERT	2026-01-05 12:17:40.882773
messagerie	INSERT	2026-01-05 12:17:43.352781
messagerie	INSERT	2026-01-05 12:17:40.886456
messagerie	INSERT	2026-01-05 12:17:40.890794
messagerie	INSERT	2026-01-05 12:17:40.894737
messagerie	INSERT	2026-01-05 12:17:40.89816

5. Vérifier le type des colonnes

Pour confirmer que le type a bien été changé en bytea :

Commande : `docker exec -it edusys-db psql -U edusys -d edusys -c "SELECT column_name, data_type FROM information_schema.columns WHERE table_schema = 'edusys' AND table_name = 'etudiant' AND column_name IN ('nom', 'prenom');"`

column_name	data_type
nom	bytea
prenom	bytea
(2 rows)	